

Sentra — User & Access Management Platform

Django REST API with authentication, roles & permissions (RBAC) + React admin frontend.
Implementation-ready spec and task plan for a single developer.

DEADLINE July 6, 2026	TEAM 1 developer (full-stack)	SUBMIT TO abdulvahab@prodevinfo.com
--------------------------	----------------------------------	--

1 • Overview & Scope

A standalone access-management application: administrators manage **users**, define **roles**, attach granular **permissions** to roles, and assign roles to users. Every API endpoint enforces role-based access control; every sensitive action is written to an **audit log**.

In scope: JWT authentication (login, signup, refresh, logout), user CRUD, role CRUD, permission catalogue, role↔permission and user↔role assignment, audit log, filter / sort / server-side pagination + Excel export on every table, React admin UI, deployment on Render free tier, public GitHub repo with README. All dependencies pinned to the latest stable versions as of July 2026.

Out of scope: SSO/OAuth providers, multi-tenancy, email verification flows (stub only), i18n.

2 • Architecture & Stack



Layer	Choice	Why
Backend	Django 6.0 + DRF 3.16 + SimpleJWT · Python 3.14	Batteries-included auth, ORM, admin; DRF viewsets keep CRUD terse
Database	PostgreSQL 18	Render free tier; SQLite for local dev via DATABASE_URL switch
Frontend	React 19 + Vite 8 + React Router 7 + TanStack Query 5	Fast dev server, cache/invalidation for CRUD, protected routes
Deploy	Render (web service + static site + Postgres)	Free tier, render.yaml blueprint = reproducible infra

Repository layout: single public monorepo — `/backend` (Django project) and `/frontend` (Vite app), root README, `render.yaml`, GitHub Actions workflow.

3 • Data Model

Custom user model from day one (`AUTH_USER_MODEL`). Roles and permissions are first-class app tables — do *not* reuse Django's group/permission tables, so the API fully owns the RBAC vocabulary.

Table	Fields	Notes
User	id (uuid pk), email (unique, login field), full_name, password, is_active, is_staff, last_login, created_at	Email login, no username. Index on email.
Role	id, name (unique), description, is_system, created_at	is_system=true (e.g. Admin) blocks delete.
Permission	id, codename (unique, "resource.action"), description	Seeded by data migration; catalogue is read-only via API.
RolePermission	role_id (fk), permission_id (fk)	M2M through-table, unique together.
UserRole	user_id (fk), role_id (fk), assigned_by (fk), assigned_at	Keeps who-assigned-what for audit.
AuditLog	id, actor_id (fk, nullable), action, target_type, target_id, changes (jsonb), ip, created_at	Append-only. Index on (created_at, actor_id).

Permission codenames (seed set): `users.view · users.create · users.edit · users.delete · roles.view · roles.manage · permissions.view · audit.view`. **Seed roles:** Admin (all), Manager (users.* + roles.view), Viewer (view-only).

4 • API Specification

All routes under `/api/v1/`. JSON only. Errors follow DRF's standard shape with a stable `detail` key. OpenAPI schema auto-generated with drf-spectacular at `/api/schema/swagger-ui/`.

List-endpoint contract (all tables): `?search=` (icontains on indexed fields), field filters via django-filter (`?role= ?is_active= ?actor= ?action= ?date_from= ?date_to=`), sorting via `?ordering=name, -created_at`, and server-side pagination `?page=&page_size=` (default 25, max 100) returning `count / next / previous / results`. Every list also exposes `GET .../export/`: a streamed **.xlsx** (openpyxl write-only mode) of the *current filter set* — same permission as the list, 10k-row cap, logged to audit.

Endpoint	Purpose	Requires
POST <code>/auth/register</code>	Signup → user with default Viewer role	anon (throttled)
POST <code>/auth/login</code>	Access token (body) + refresh token (httpOnly cookie)	anon (throttled)
POST <code>/auth/refresh</code> · POST <code>/auth/logout</code>	Rotate refresh (blacklist old) · blacklist + clear cookie	refresh cookie
GET <code>/auth/me</code>	Current user + roles + flattened permission codenames	authenticated
GET POST <code>/users</code> · GET PATCH DELETE <code>/users/{id}</code>	User CRUD (delete = soft-deactivate); full list contract: search, filters, sort, pagination.	users.*
PUT <code>/users/{id}/roles</code>	Replace a user's role set (list of role ids)	roles.manage
GET POST <code>/roles</code> · GET PATCH DELETE <code>/roles/{id}</code>	Role CRUD; system roles are delete-protected	roles.view / roles.manage
PUT <code>/roles/{id}/permissions</code>	Replace a role's permission set	roles.manage
GET <code>/permissions</code>	Read-only permission catalogue	permissions.view
GET <code>/audit-logs</code>	Full list contract: filters, sort, server-side pagination	audit.view
GET <code>/users/export</code> · <code>/roles/export</code> · <code>/audit-logs/export</code>	Excel (.xlsx) download of the current filter set	same as list

5 • Authentication & Authorization Design

Tokens. SimpleJWT: access token 15 min, kept in JS memory only (never localStorage). Refresh token 7 days in an `httpOnly; Secure; SameSite=None` cookie with rotation + blacklist on every refresh. Axios interceptor retries a 401 once via `/auth/refresh`, then redirects to login.

Enforcement — backend is the source of truth. A single DRF permission class `HasPermission("users.edit")` resolves the user's permission set (user → roles → permissions, cached per-request) and is declared on every viewset action map. No endpoint ships without an explicit permission — default DRF policy set to `IsAuthenticated` so nothing is accidentally public.

Frontend gating is UX only. `/auth/me` returns flattened codenames; a `<Can perm="roles.manage">` component hides buttons/routes the user can't use. Guard rails: you cannot deactivate yourself or remove your own Admin role.

6 • Security Checklist

- | | |
|--|---|
| ☐ Argon2 password hashing + Django password validators | ☐ All secrets via env vars; <code>DEBUG=False</code> in prod; <code>.env.example</code> committed, <code>.env</code> ignored |
| ☐ Refresh-token rotation with blacklist; logout invalidates | ☐ ORM only — no raw SQL; serializer validation on every write |
| ☐ Login/register throttling (5/min per IP) via DRF throttles | ☐ Object-level checks (self-lockout guards, system-role protection) |
| ☐ Strict CORS allow-list (frontend origin only) + CSRF-safe cookie config | ☐ Generic error messages on login (no user enumeration) |
| ☐ Security headers: HSTS, nosniff, X-Frame-Options DENY, Referrer-Policy | ☐ pip-audit + npm audit in CI; Dependabot enabled |
-

7 • Performance & Code Quality

Performance: cursor-friendly page-number pagination (25/page); `prefetch_related("roles__permissions")` on user list to kill N+1; DB indexes on email, codename, audit timestamps; gzip via whitenoise; Vite code-splitting per route; TanStack Query caching with invalidation on mutations.

Quality gates (CI on every push): backend — ruff + black, pytest with ≥80% coverage on auth/RBAC paths; frontend — ESLint + Prettier, vitest for the permission-gate and auth hooks. Conventional commits; PR-less trunk workflow is fine for one dev, but keep commits atomic.

Tests that matter most: permission matrix test (each seed role × each endpoint → expected 200/403), token refresh/rotation flow, self-lockout guards, audit-log write on every mutation.

8 • Frontend Architecture

```
frontend/src/
├─ api/           axios + endpoints
├─ auth/          AuthProvider, useAuth,
│                 Can, ProtectedRoute
├─ pages/
│   ├─ Login      └─ Dashboard
│   ├─ Users      └─ Roles
│   └─ AuditLog
├─ components/    table, modal,
│                 toast, form fields
└─ App.jsx        router
```

Routing: public `/login` , `/signup` ; everything else behind `ProtectedRoute` which blocks until `/auth/me` resolves.

Route-level permission requirements (e.g. Audit Log needs `audit.view`).

DataTable pattern (Users, Audit Log): one shared component — toolbar with search + filter selects, sortable column headers (maps to `?ordering=`), server-side pagination footer, and an **Export to Excel** button that calls the list's `/export/` endpoint with the active filters. All table state lives in the URL query string so views are shareable and back-button safe.

Create account: full name, email, password + confirm with live match/strength validation mirroring Django's validators; new accounts get the Viewer role and land on the dashboard signed in.

UX standards: optimistic-free (server-confirmed) mutations with toast feedback, skeleton loading states, empty states with a primary action, destructive actions behind confirm dialogs, inline form validation mirroring API errors.

See the **interactive prototype** shipped with this brief for the exact screens: login, dashboard, users, roles & permissions matrix, role assignment, audit log.

9 • Deployment on Render (free tier)

One `render.yaml` blueprint defines: **web service** (Python, `gunicorn config.wsgi` , build runs `migrate + collectstatic + seed` command), **static site** (Vite build, SPA rewrite rule `/* → /index.html`), **PostgreSQL** free instance. Env vars: `SECRET_KEY` , `DATABASE_URL` , `ALLOWED_HOSTS` , `CORS_ALLOWED_ORIGINS` , `VITE_API_URL` .

Free-tier caveats to note in README: web service sleeps after 15 min idle (first request ~30s cold start); free Postgres expires after 30 days. Seed command creates the demo admin (`admin@sentra.dev`) and sample data so reviewers can log in immediately.

10 • Task Breakdown

Phase	Tasks	Done means
Phase 1	Repo + tooling scaffold (ruff, black, ESLint, CI skeleton) → custom User model → Role/Permission/UserRole/AuditLog models + migrations → seed data migration → SimpleJWT auth endpoints (login, refresh w/ cookie rotation, logout, register, me)	Login → refresh → logout round- trip passes in pytest
Phase 2	HasPermission class + audit-log middleware → users / roles / permissions / audit viewsets → assignment endpoints → list contract (django-filter, ordering, pagination) + xlsx export endpoints → permission- matrix test suite → throttling, CORS, security headers → Swagger docs	API complete; matrix tests green; ≥80% coverage
Phase 3	React scaffold + AuthProvider + axios interceptors → Login/Signup → app shell + protected routes → Users page (DataTable: filters, sort, server-side pagination, Excel export, create/edit modal, deactivate, assign roles) → Roles page (CRUD + permission matrix) → Audit Log page → Dashboard	Full flows work against local API; Can-gating verified per role
Phase 4	render.yaml + deploy all three services → prod smoke test with seeded roles → README (setup, env vars, API docs link, demo credentials, architecture notes) → final CI green → email repo + live URLs to abdulvahab@prodevinfo.com	Reviewer can log in on the live URL within 1 minute of opening README

Buffer rule: if Phase 3 slips, cut the Dashboard stat cards and Signup page (login-only is acceptable) — never cut tests, security config, or deployment.

11 • Acceptance Checklist

- ☐ Public GitHub repo with README (setup in ≤5 commands, demo credentials, live URLs, API docs link)
- ☐ Deployed on Render: live frontend + API + Swagger UI reachable
- ☐ Three seed roles behave differently in the UI and get 403s at the API where expected
- ☐ CI green: lint + tests + coverage gate on latest main
- ☐ Emailed to **abdulvahab@prodevinfo.com** on or before **July 6, 2026**